

I GPT: generative pretrained transformer(only?)

- Generation can be done not just by transformer, but other models.



Seedance 1.0: Exploring the Boundaries of Video Generation Models

ByteDance Seed

Abstract

Notable breakthroughs in **diffusion modeling** have propelled rapid improvements in video generation, yet current foundational model still face critical challenges in simultaneously balancing prompt following, motion plausibility, and visual quality. In this report, we introduce **Seedance 1.0**, a high-performance and inference-efficient video foundation generation model that integrates several core technical improvements: (i) multi-source data curation augmented with precision and meaningful video captioning, enabling comprehensive learning across diverse scenarios; (ii) an efficient architecture design with proposed training paradigm, which allows for natively supporting multi-shot generation and jointly learning of both text-to-video and image-to-video tasks. (iii) carefully-optimized post-training approaches leveraging fine-grained supervised fine-tuning, and video-specific RLHF with multi-dimensional reward mechanisms for comprehensive performance improvements; (iv) excellent model acceleration achieving 10× inference speedup through multi-stage distillation strategies and system-level optimizations. Seedance 1.0 can generate a 5-second video at 1080p resolution only with 41.4 seconds (NVIDIA-L20). Compared to state-of-the-art video generation models, Seedance 1.0 stands out with high-quality and fast video generation having superior spatiotemporal fluidity with structural stability, precise instruction adherence in complex multi-subject contexts, native multi-shot narrative coherence with consistent subject representation. Seedance 1.0 is now accessible on **Volcano Engine**^α.

Official Page: <https://seed.bytedance.com/seedance>

^a**Model ID:** Doubao-Seedance-1.0-pro

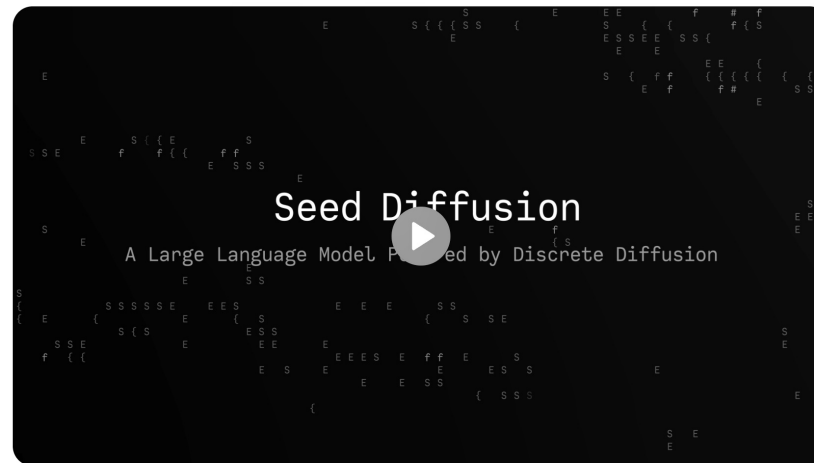
July 31, 2025

Seed Diffusion Preview

A large scale language model based on discrete-state diffusion, specializing in code generation, achieves an inference speed of 2,146 token/s, a 5.4x improvement over autoregressive models of comparable size.

[Read Paper ↗](#)

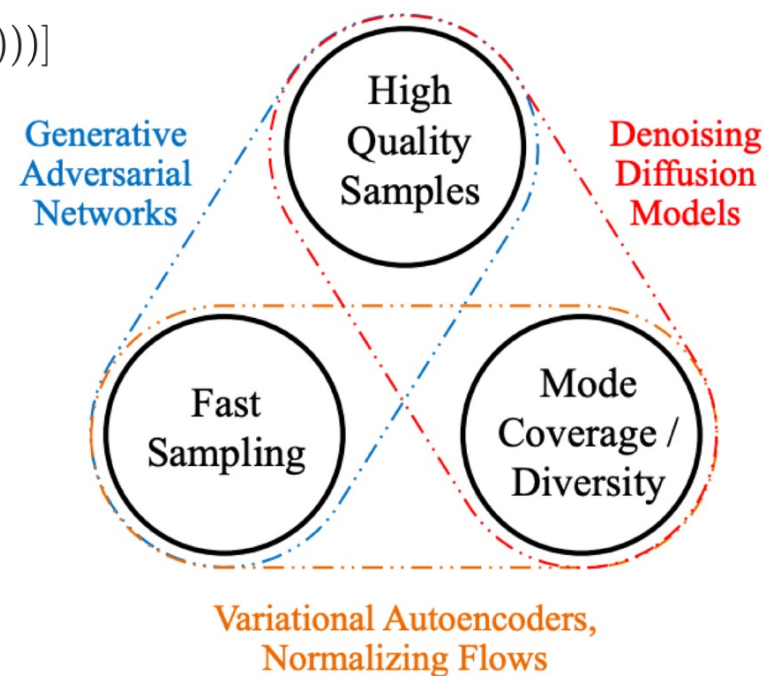
Try Now ↗



Generative Models Overview

When evaluating generative frameworks, we encounter the "Generative Trilemma". sample fidelity, fast sampling, and mode coverage (diversity).

- **GANs**: optimize $V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$
 D represents the **Discriminator**, and G is the **Generator** (mapping noise z to data space x).
- **VAEs**: maximize $\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p(z))$, $q_\phi(z|x)$ is the **Encoder** (mapping data to a latent distribution), $p_\theta(x|z)$ **Decoder** (reconstructing data from the latent space), $p(z)$ is the Prior Distribution of the latent variables.
- **Diffusion Models**: utilize a learnable reverse Markov chain
 $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)$, $p_\theta(x_{t-1}|x_t)$ represents the learned Gaussian transition kernel at each denoising step.



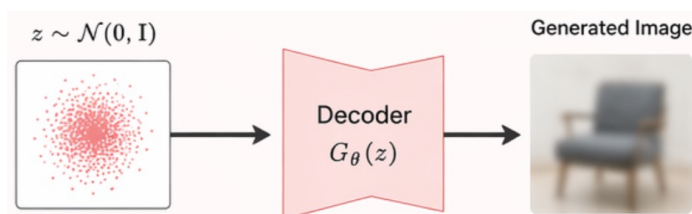
Diffusion model has multiple steps of iterative sampling, while GAN/VAEs generate sample in 1 step.

Image source: <https://javierisolisgarcia.com/posts/ddpm/>

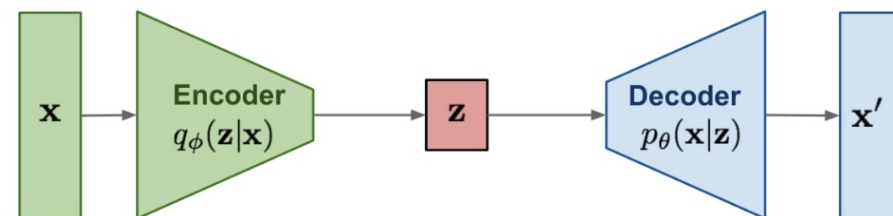
Iterative Refinement

Diffusion models introduce **iterative refinement**. Instead of one giant leap from a prior to the data distribution, diffusion models break the generation process into many small, reversible steps. By decomposing the transformation into a Markov chain of T steps (e.g., 1000) each step becomes tractable and easier for neural networks (such as U-Net) to learn.

- VAE: one large step of transformation that can be hard for a neural network to fit for diverse samples.



VAE: maximize variational lower bound



- Diffusion: many small steps of transformation that can be easier for a neural network to fit.



Diffusion models:
Gradually add Gaussian noise and then reverse

each step done
by a neural network

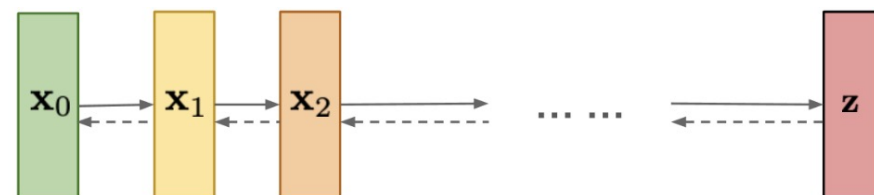


Image source: <https://javalierisgarcia.com/posts/ddpm/>

The Forward Stochastic Process

Physical Intuition: Thermodynamics and Entropy

- Diffusion Models stems from non-equilibrium thermodynamics.
- Consider a **drop of ink (low entropy)** diffusing in a container of water; the system naturally evolves toward a state when the ink is **uniformly dispersed (high entropy)**.
- Simulate this **forward process** by systematically adding Gaussian noise to the current state x_{t-1} to get the next state x_t , eventually transforming data into isotropic noise (standard Gaussian, *which is the distribution with the highest entropy given mean and standard deviation*).

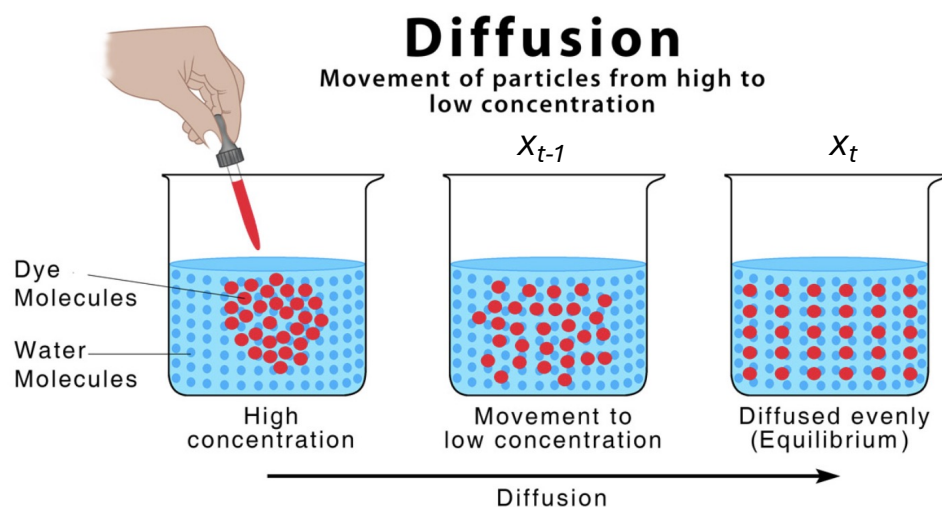


Image source: <https://www.sciencefacts.net/diffusion.html>

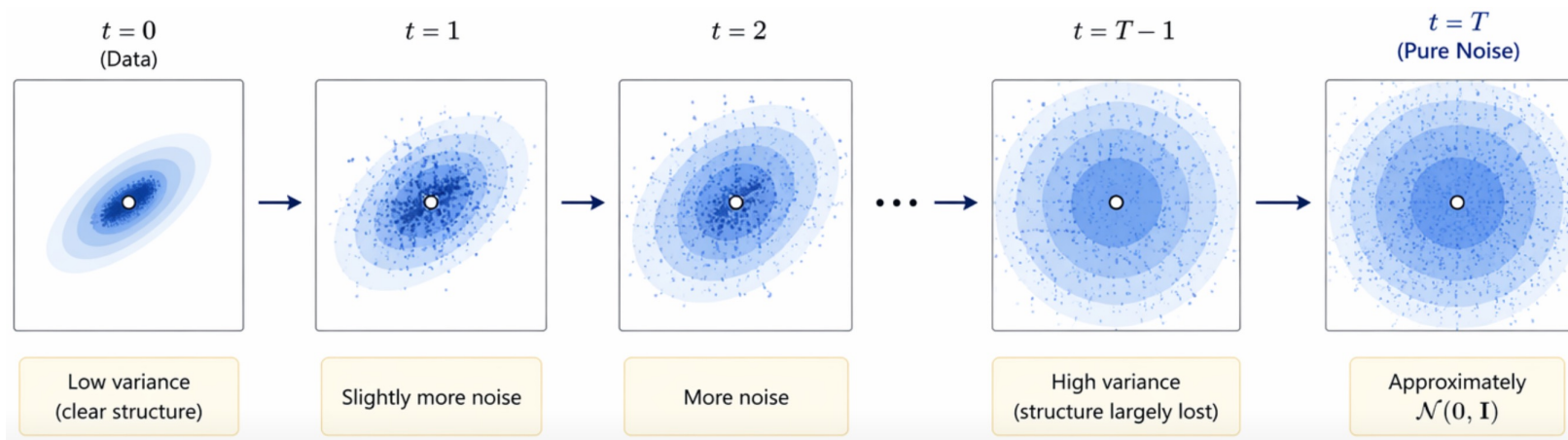
Markovian Degradation: The forward process $q(x_t|x_{t-1})$ acts as a Markovian information decay, incrementally replacing structured signal (something informative, such as location/shape/scale) with isotropic uncertainty (something non-informative).

Mathematical Formulation of the Forward Process

The forward process q is a predefined Markov chain that adds Gaussian noise according to a variance schedule $\beta_1, \dots, \beta_T$. The **transition kernel** is defined as:

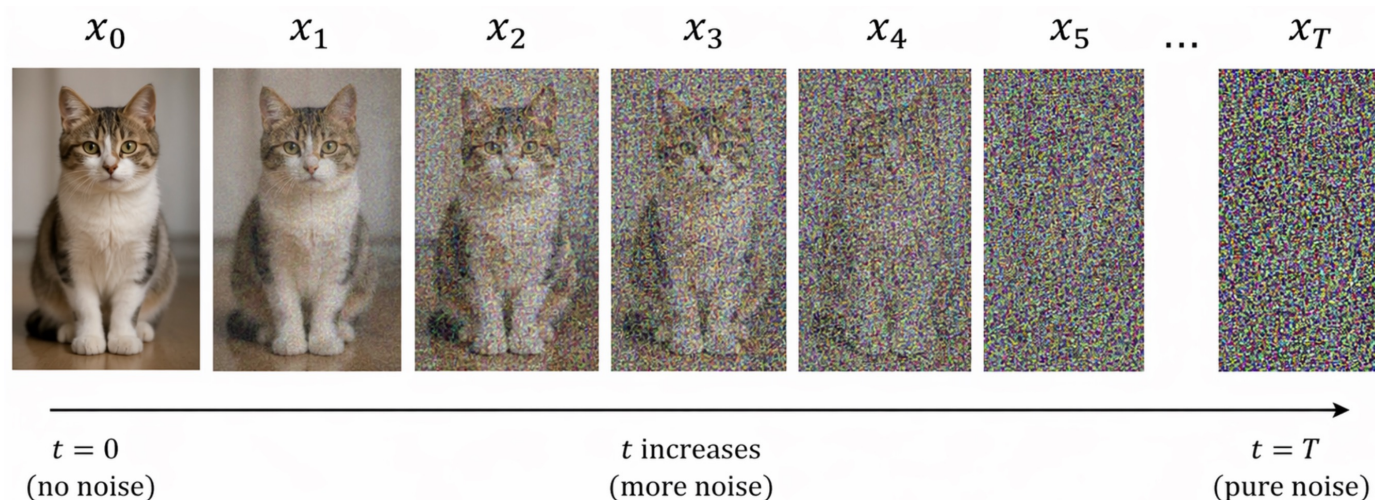
$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}) \quad (\text{If } \beta_t \rightarrow 0, \text{ it collapse to just } x_{t-1})$$

- As t increases, the original structure of x_0 is dispersed. The term $\sqrt{1 - \beta_t}$ acts as a scaling factor to prevent the variance from exploding during the noise injection steps (sudden changes are harder for a neural network to learn).
- Visualizing the distribution $q(x_t | x_{t-1})$ in 2D:



Analyzing Stochastic Degradation

In the forward process, the original signal x_0 is systematically *attenuated*. As $t \rightarrow T$, the influence of x_0 vanishes, and the distribution converges to a standard normal distribution $\mathcal{N}(0, \mathbf{I})$, regardless of the initial input x_0 .



This process contains no learnable parameters; it serves as a hypothetical “destruction schedule” that provides the intuition for a learnable reverse process (learn to generate x_0 from standard normal).

Image source: Openai

Variance Schedules

Define $\alpha_t = 1 - \beta_t$ and its cumulative product $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$

β_t : **Variance Schedule** $\alpha_t = 1 - \beta_t$: **Signal Retention**

The choice of β_t and thus $\bar{\alpha}_t$ is crucial.

- Linear schedule: the signal drops to zero too early.
- Cosine schedule: preserves the signal for longer and avoids the "abrupt collapse" near the end.

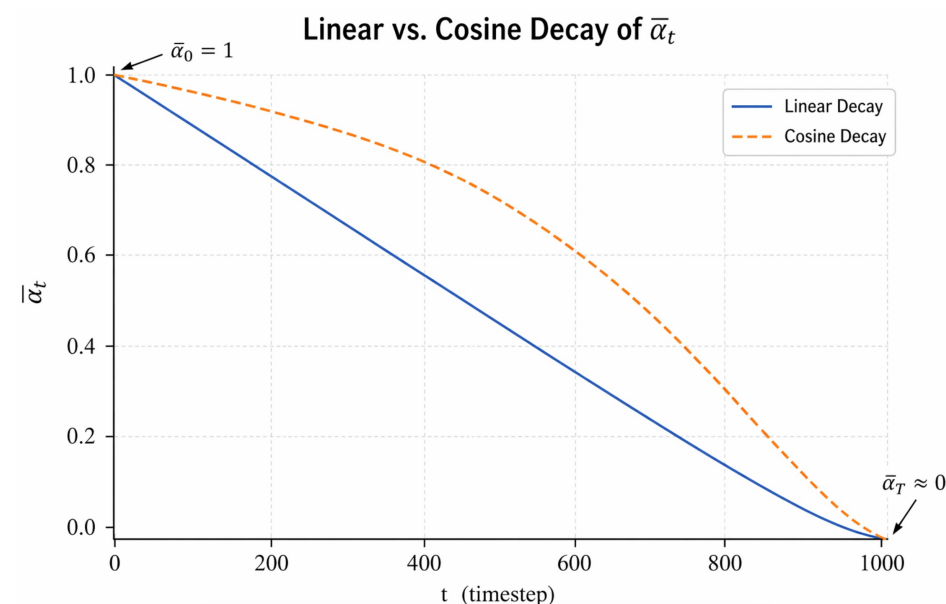


Image source: Openai

Deriving a key property of forward noising

expand x_t by substituting the expression x_{t-1} : $x_{t-1} = \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}$

Substituting this into the equation for x_t : $x_t = \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1 - \alpha_t}\epsilon_{t-1}$

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$$

The sum of two independent Gaussians is also a Gaussian: $\mathcal{N}(0, \sigma_1^2\mathbf{I}) + \mathcal{N}(0, \sigma_2^2\mathbf{I}) \sim \mathcal{N}(0, (\sigma_1^2 + \sigma_2^2)\mathbf{I})$

So the two noise terms are: 1. $\sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} \sim \mathcal{N}(0, \alpha_t(1 - \alpha_{t-1})\mathbf{I})$

2. $\sqrt{1 - \alpha_t}\epsilon_{t-1} \sim \mathcal{N}(0, (1 - \alpha_t)\mathbf{I})$

that's why the squared roots
in the noise schedule

The combined variance is: $\sigma_{combined}^2 = \alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = \alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}$

Thus, we can rewrite the two noise terms as a single Gaussian $\sqrt{1 - \alpha_t\alpha_{t-1}}\epsilon$

We obtain:

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\epsilon = \sqrt{\bar{\alpha}_t}x_{t-2} + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Bayesian Inference for the Reverse Process

Data generation process $p_{\theta}(x_{t-1}|x_t)$ is to reverse the forward process $q(x_t|x_{t-1})$. Learn a model θ to take a noisy sample x_t and estimate what it looked like x_{t-1} one step earlier. Since each noising step is *infinitesimal* (a tiny step of change), a reverse transition can also be modeled as a Gaussian distribution, where the network predicts the noise's mean $\mu_{\theta}(x_t, t)$ and variance $\Sigma_{\theta}(x_t, t)$.

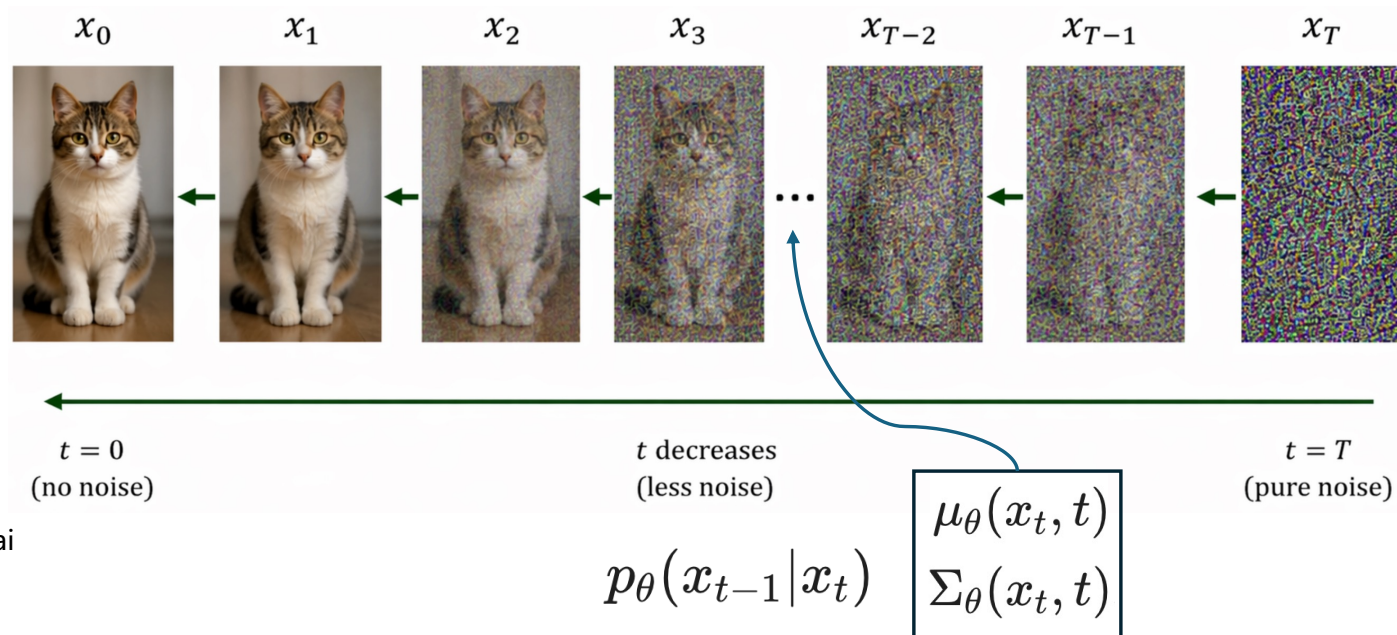


Image source: Openai

Objectives and Training

1. We want to maximize the likelihood of the data x_0 : $\min_{\theta} \mathcal{L}_{NLL} = -\log p_{\theta}(x_0)$
2. The marginal likelihood is intractable. Introduce a variational distribution $q(x_{1:T} | x_0)$ and apply Jensen's Inequality ($\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$) to get an upper bound:

$$-\log p_{\theta}(x_0) = -\log \int q(x_{1:T}|x_0) \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} dx_{1:T}$$

3. Define the Variational Upper Bound:

$$\mathcal{L}_{VLB} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{0:T})} \right] \geq -\log p_{\theta}(x_0)$$

4. Expand the terms:

$$\mathcal{L}_{VLB} = \mathbb{E}_q \left[\log \frac{\prod_{t=1}^T q(x_t|x_{t-1})}{p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)} \right]$$

5. Group terms into KL Divergences:

$$\mathcal{L}_{VLB} = \mathbb{E}_q \left[\log \frac{q(x_T|x_0)}{p(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)} - \log p_{\theta}(x_0|x_1) \right]$$

Objectives and Training

Objective Function: Variational Lower Bound

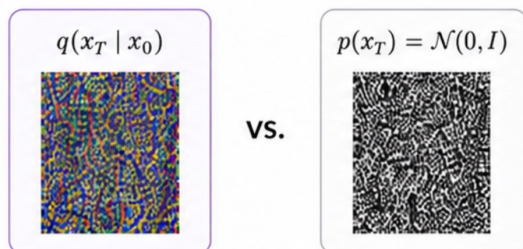
We optimize the model by maximizing the Variational Lower Bound (ELBO) of the log-likelihood.

$$\text{ELBO (to be minimized)} \quad \mathcal{L} = \underbrace{\mathbb{E}_q \left[D_{KL}(q(x_T | x_0) \parallel p(x_T)) \right]}_{\mathcal{L}_T \text{ (prior loss)}} + \underbrace{\sum_{t=1}^T D_{KL}(q(x_{t-1} | x_t, x_0) \parallel p_\theta(x_{t-1} | x_t))}_{\mathcal{L}_t \text{ (denoising loss)}} - \underbrace{\log p_\theta(x_0 | x_1)}_{\mathcal{L}_0 \text{ (reconstruction loss)}}$$

1. $\mathcal{L}_T$ (Prior Loss)

Matches the final noisy sample x_T to the prior $p(x_T)$ (usually $\mathcal{N}(0, I)$).

$$\mathcal{L}_T = D_{KL}(q(x_T | x_0) \parallel p(x_T))$$

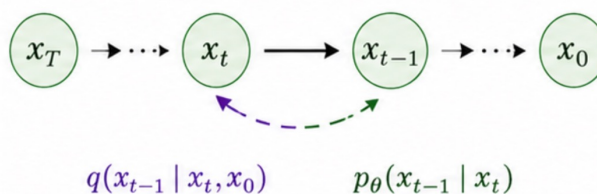


Encourages the forward process to end in pure Gaussian noise.

2. $\mathcal{L}_t$ (Denoising Loss)

Sum of KL divergences at each timestep t between the true posterior and the model posterior.

$$\mathcal{L}_t = \sum_{t=1}^T D_{KL}(q(x_{t-1} | x_t, x_0) \parallel p_\theta(x_{t-1} | x_t))$$

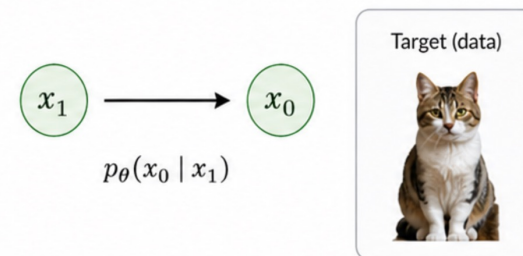


Trains the model to denoise step-by-step and match the true reverse process.

3. $\mathcal{L}_0$ (Reconstruction Loss)

Negative log-likelihood of the data under the model at the final step.

$$\mathcal{L}_0 = -\log p_\theta(x_0 | x_1)$$



Measures how well the model can reconstruct the original data from x_1 .

Bayesian Challenges

The true reverse distribution $q(x_{t-1}|x_t)$ is intractable because it requires knowledge of the marginal distribution $q(x)$, which involves an integral over the entire data space. We cannot compute this directly. Instead, we use a neural network p_θ to approximate this unknown distribution. This challenge is the central motivation for the training objectives we will derive next.

$$q(x_{t-1} | x_t) = \frac{q(x_t | x_{t-1}) q(x_{t-1})}{q(x_t)}$$

Forward transition
known

Marginal of x_{t-1}
intractable

Marginal of x_t
intractable

Computing $q(x_t)$ requires:

$$\begin{aligned} q(x_t) &= \int q(x_t | x_{t-1}) q(x_{t-1}) dx_{t-1} \\ &= \text{Integral over all } x_{t-1} \in \mathbb{R}^d \quad (\text{intractable}) \end{aligned}$$

Intractable
requires integral
over all x_t

Summary:

By Bayes' Rule,

$$q(x_{t-1} | x_t) = \frac{q(x_t | x_{t-1}) q(x_{t-1})}{q(x_t)}$$

but $q(x_t)$ is intractable.

Tractable Posterior via Clean data Conditioning

$q(x_{t-1}|x_t)$ is intractable but the posterior conditioned on x_0 , $q(x_{t-1}|x_t, x_0)$ is tractable!
By using x_0 as a reference, the reverse step can be solved using Bayes' Rule as a product of known Gaussian forward kernels:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

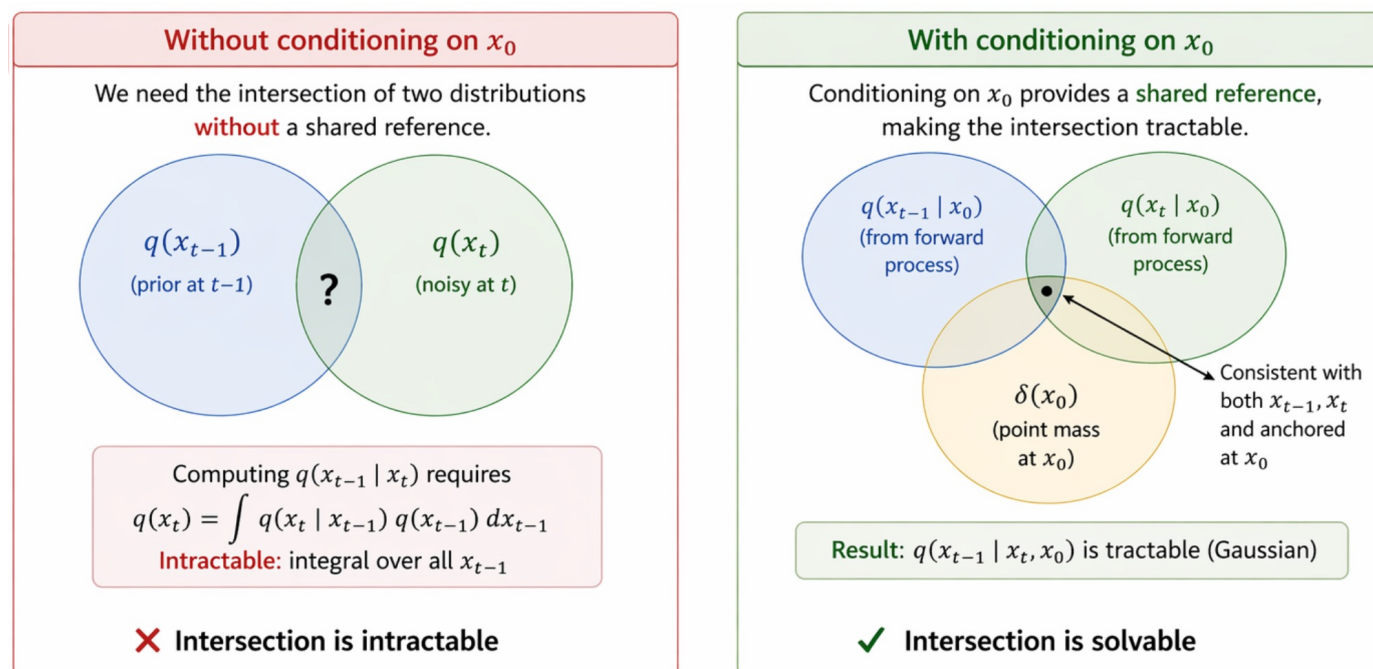


Image source: Openai

Deriving the Tractable Posterior Mean $\tilde{\mu}_t$

We want to find $q(x_{t-1}|x_t, x_0)$. Using Bayes' Rule and the Markov property:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)} \propto q(x_t|x_{t-1})q(x_{t-1}|x_0)$$

Substituting the Gaussian forms (ignoring constants that do not depend on x_{t-1}):

$$\propto \exp\left(-\frac{1}{2}\left(\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{1 - \bar{\alpha}_{t-1}}\right)\right)$$

Expanding the squares and collecting terms for x_{t-1} :

$$\propto \exp\left(-\frac{1}{2}\left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right)x_{t-1}^2 - 2\left(\frac{\sqrt{\alpha_t}x_t}{\beta_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}x_0}{1 - \bar{\alpha}_{t-1}}\right)x_{t-1} + C\right]\right)$$

For a Gaussian $\mathcal{N}(x; \mu, \sigma^2)$, the exponent is $-\frac{1}{2\sigma^2}(x^2 - 2\mu x)$. Thus, the mean $\tilde{\mu}_t$ is the ratio of the coefficient of x_{t-1} to the coefficient of x_{t-1}^2 :

Time-dependent interpolation between x_t and x_0

$$\tilde{\mu}_t(x_t, x_0) = \frac{\frac{\sqrt{\alpha_t}}{\beta_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}x_0}{\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}}$$
$$\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$$
$$\beta_t = 1 - \alpha_t$$

Image source: Openai

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0$$

Substituting x_0 to derive μ_θ

In the reverse process (inference), x_0 is unknown. We use the forward marginal formula to express x_0 in terms of x_t and the noise ϵ :

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)$$

Substitute this into the formula for $\tilde{\mu}_t$:

$$\mu_\theta(x_t, t) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \left(\frac{1}{\sqrt{\bar{\alpha}_t}}x_t - \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}}\epsilon_\theta(x_t, t) \right)$$

Combining the terms for x_t :

$$\text{Coeff of } x_t = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{(1 - \bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} = \frac{\alpha_t - \bar{\alpha}_t + \beta_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)}$$

For the noise term:

$$\text{Coeff of } \epsilon_\theta = -\frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t\sqrt{1 - \bar{\alpha}_t}}{(1 - \bar{\alpha}_t)\sqrt{\alpha_t\bar{\alpha}_{t-1}}} = -\frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t}}$$

Factoring out $\frac{1}{\sqrt{\alpha_t}}$:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_\theta(x_t, t) \right)$$

Deriving the Reverse Mean

By expanding the Gaussian exponents and completing the square, we find that the mean $\tilde{\mu}_t$ of the tractable posterior $q(x_{t-1}|x_t, x_0)$ is a linear combination of x_t and x_0 :

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0$$

During inference, x_0 is unknown. However, we can express x_0 in terms of x_t and the noise ε added to it. Substituting this into the mean formula reveals a striking result: predicting the reverse mean is mathematically equivalent to predicting the noise ε :

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

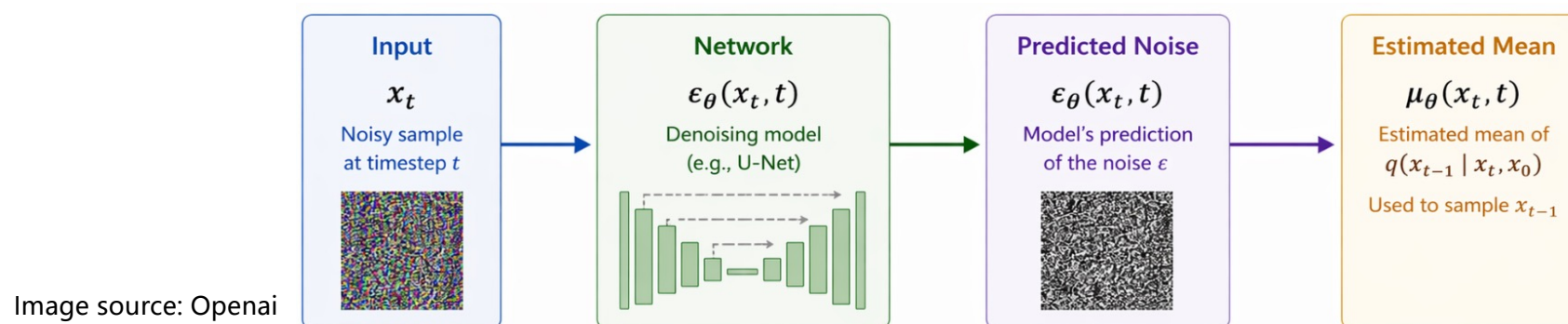


Image source: Openai

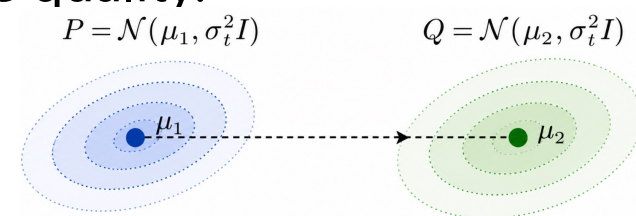
The Simplified Loss

Ho et al. (2020) discovered that the KL Divergence between two Gaussians with fixed variances simplifies to an L2 distance between their means. Crucially, they found that dropping the weighting coefficients yields even better sample quality:

$$\text{Let } P = \mathcal{N}(\mu_1, \Sigma_1) \\ Q = \mathcal{N}(\mu_2, \Sigma_2)$$

Both are d-dimensional Gaussian distributions.

$$D_{KL}(P||Q) = \frac{1}{2} \left[\underbrace{\log \frac{|\Sigma_2|}{|\Sigma_1|}}_{\text{Log-determinant term}} - \underbrace{d}_{\text{Dimension term}} + \underbrace{\text{tr}(\Sigma_2^{-1} \Sigma_1)}_{\text{Trace term}} + \underbrace{(\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1)}_{\text{Quadratic (mean) term}} \right]$$



$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 \mathbf{I})$$

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon \right)$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right)$$

$$D_{KL}(\mathcal{N}(\mu_1, \sigma_t^2 \mathbf{I}) || \mathcal{N}(\mu_2, \sigma_t^2 \mathbf{I})) \\ = \frac{1}{2\sigma_t^2} \|\mu_1 - \mu_2\|_2^2$$

Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." NIPS 2020: 6840-6851.

(i) Log-Determinant Ratio

Since $\Sigma_1 = \Sigma_2 = \sigma_t^2 \mathbf{I}$, their determinants are equal.

$$\log \frac{|\Sigma_2|}{|\Sigma_1|} = \log \frac{|\sigma_t^2 \mathbf{I}|}{|\sigma_t^2 \mathbf{I}|} = \log(1) = 0$$



(ii) Trace and Dimension Terms

Since $(\sigma_t^2 \mathbf{I})^{-1} (\sigma_t^2 \mathbf{I}) = \mathbf{I}$,

$$\text{tr}((\sigma_t^2 \mathbf{I})^{-1} (\sigma_t^2 \mathbf{I})) = \text{tr}(\mathbf{I}_{d \times d}) = d$$

Thus, the dimension terms cancel out:

$$-d + d = 0$$



(iii) Quadratic Term (The L_2 Distance)

The remaining term becomes:

$$\begin{aligned} & (\mu_2 - \mu_1)^T (\sigma_t^2 \mathbf{I})^{-1} (\mu_2 - \mu_1) \\ &= \frac{1}{\sigma_t^2} (\mu_2 - \mu_1)^T (\mu_2 - \mu_1) \\ &= \frac{1}{\sigma_t^2} \|\mu_1 - \mu_2\|_2^2 \end{aligned}$$

Squared L_2 distance between means, scaled by $1/\sigma_t^2$.

Implication for Diffusion Model Training

The denoising matching term in the ELBO:

$$D_{KL}(q(x_{t-1} | x_t, x_0) || p_\theta(x_{t-1} | x_t))$$

becomes the **squared error** between the true posterior mean and the model's predicted mean.

$$\Rightarrow \|\mu_{true} - \mu_\theta\|_2^2$$



With the common parameterization, this is equivalent to **noise prediction**:

$$\Rightarrow \|\epsilon - \epsilon_\theta(x_t, t)\|_2^2$$



Therefore, minimizing the KL term is equivalent to minimizing an L_2 loss.

U-Net and Time Embeddings

The standard architecture for diffusion model is a U-Net with skip connections.

- Skip connections allow the network to preserve fine-grained spatial details while understanding global structure through the bottleneck.
- Since the denoising logic depends on the current noise level, we must inject a Time Embedding (often sinusoidal) into the network to condition its predictions on t .

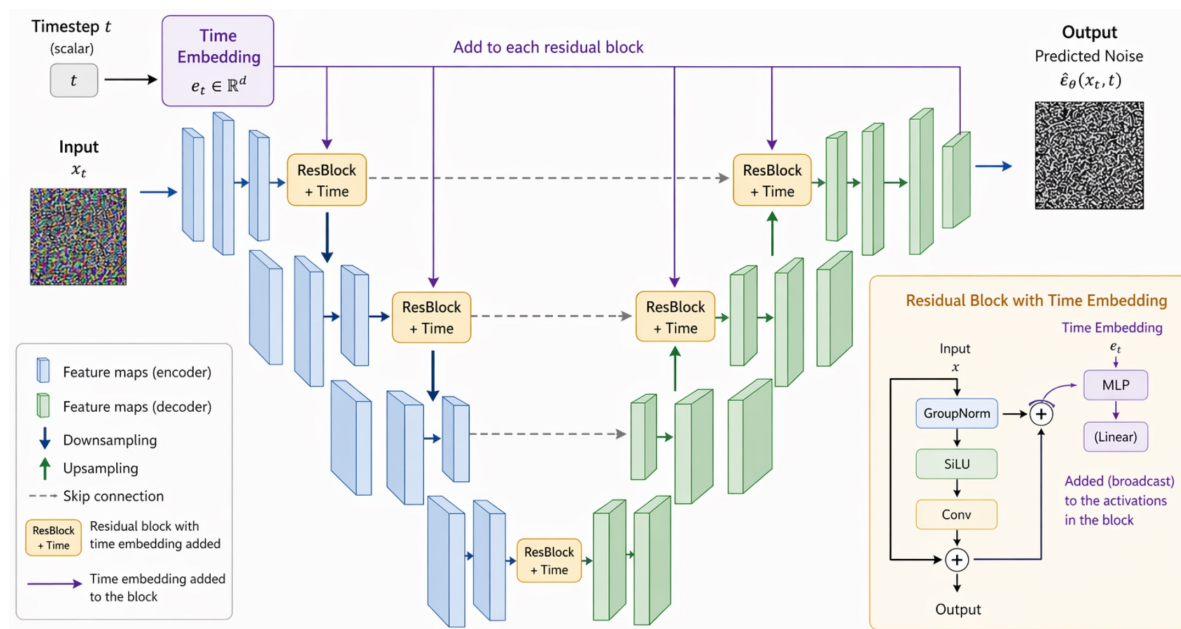


Image source: Openai

Training Algorithm

The training procedure is efficient: each iteration only requires one forward noising step.

Algorithm 1 (Training)

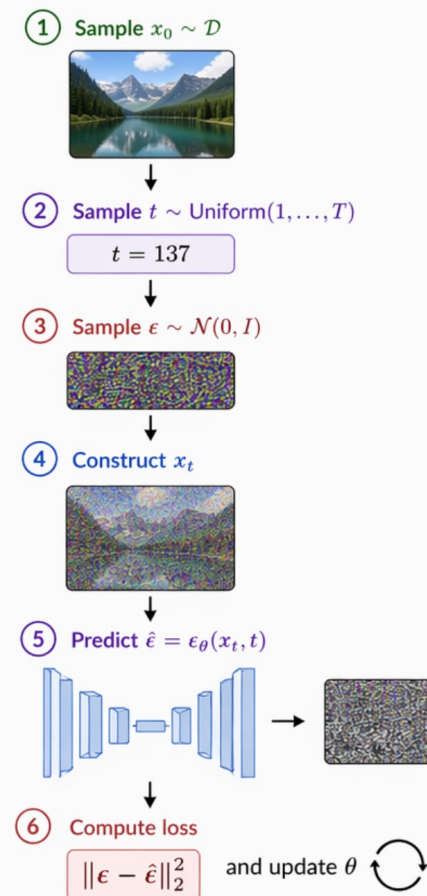
Input: dataset $\mathcal{D}$, diffusion steps T , noise schedule $\{\bar{\alpha}_t\}_{t=1}^T$,
model $\epsilon_\theta(x_t, t)$ with parameters θ

repeat

- 1: Sample a clean image $x_0 \sim \mathcal{D}$. # data
- 2: Sample a random time step $t \sim \text{Uniform}(1, \dots, T)$. # timestep
- 3: Sample Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$. # noise
- 4: Construct the noisy image
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$
 # forward noising
- 5: Predict the noise using the network
$$\hat{\epsilon} = \epsilon_\theta(x_t, t)$$
 # model prediction
- 6: Compute the simple loss
$$\mathcal{L} = \|\epsilon - \epsilon_\theta(x_t, t)\|_2^2$$
 # simple (unweighted) loss
- 7: Update network weights θ to minimize $\mathcal{L}$ (e.g., via Adam). # optimization step

until converged

Image source: Openai



Sampling Algorithm - Generation

The sampling (reverse) procedure: each iteration requires one backward denoising step.

Algorithm 2 (Sampling / Generation)

Input: trained noise prediction model $\epsilon_\theta(x_t, t)$,
noise schedule $\{\bar{\alpha}_t, \sigma_t^2\}_{t=1}^T$

Output: generated sample x_0

1: Sample $x_T \sim \mathcal{N}(0, I)$ # start from pure noise

2: for $t = T, T-1, \dots, 1$ do

2a: Predict noise $\hat{\epsilon} = \epsilon_\theta(x_t, t)$

2b: Compute mean (posterior mean)

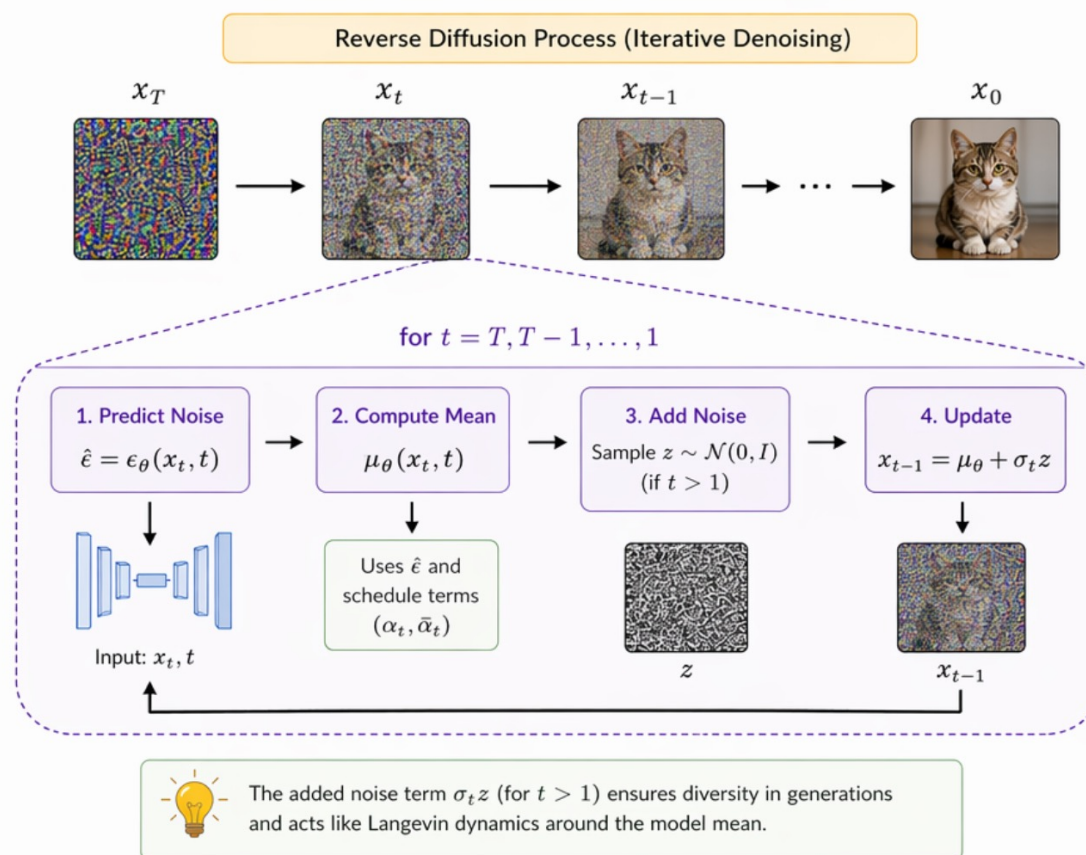
$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right)$$

2c: Sample noise $z \sim \mathcal{N}(0, I)$ if $t > 1$,
else set $z = 0$ # Langevin-like noise
for diversity

2d: Update $x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$

end for # repeat until $t = 1$

3: Return x_0 # recovered clean image



Case Study - CIFAR-10 Progressive Generation

Observing the progressive generation on CIFAR-10 reveals how the model prioritize features.

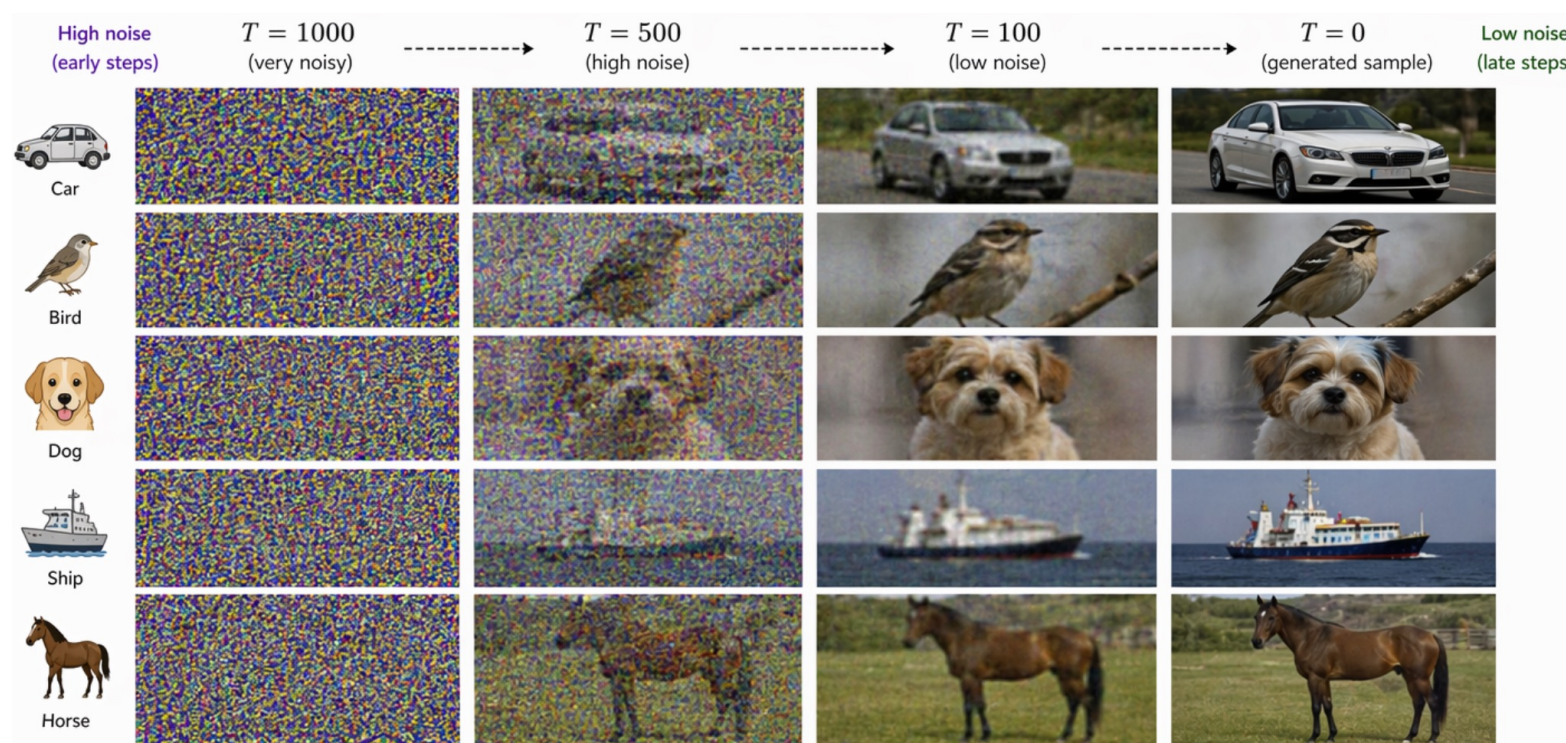


Image source: Openai

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (T/F): training a diffusion model can be reduced to letting a neural network predict the noise ϵ added to x_0 to obtain x_t .
 - Q2 (T/F): the sum of two Gaussian variables is another Gaussian variable.
 - Q3 (T/F): during inference time of a diffusion model, the starting point x_0 should be known as the the distribution $q(x_{t-1}|x_t, x_0)$ requires it.
 - A: T; T; F